Domain Adaptation with Small Language Models for Shakespearean Text Understanding

Group Number: *[Insert Group Number]*
CSCI433/933 Machine Learning Algorithms and Applications
Student Names and Numbers: *[Insert Details]*

Abstract—This report presents a Shakespeare-aware language system that helps a user with no prior exposure to Shakespeare ask questions about *Hamlet*, *Macbeth*, and *Romeo and Juliet* and receive accurate, evidence-grounded explanations. We treat the task as one of domain adaptation: rather than pretraining or full fine-tuning, we adapt a compact pretrained model, Gemma 3 4B running locally via Ollama, using retrieval-augmented generation (RAG) and prompt design. The provided structured corpus is prepared at two granularities, scene-level chunks and speaker-level utterances, with each retrieval unit enriched by a modern-English scene summary and keywords and indexed by a MiniLM sentence embedding. Three systems are compared: a no-retrieval baseline, a standard RAG pipeline, and an enhanced pipeline that adds cross-encoder reranking and scene-level diversity. The system supports four interaction modes, concept explanation, contextual question answering, evidence retrieval, and clearly labelled stylised generation, and always displays retrieved evidence alongside its answers. We evaluate on fifteen questions (five instructor-provided, ten group-designed) using a five-criterion rubric. Retrieval relevance is consistently strong, while the analysis isolates the generation stage as the primary bottleneck and the main target for future improvement.

I. Introduction and Problem Formulation

Recent progress in natural language processing has been driven largely by very large foundation models. In practice, however, many organisations cannot afford, govern, or justify hosting such models for narrow, bounded tasks. This has motivated growing interest in *small language models* (SLMs): compact, deployable models that can be adapted to a specific domain under realistic constraints on cost, privacy, latency, and maintainability. The central question this project addresses is therefore not how to build the largest possible system, but how to make a small, constrained model reliable and useful on a well-defined domain task.

A. The Task

We build a Shakespeare-aware language system that lets a user with little or no prior exposure to Shakespeare interrogate three plays, *Hamlet*, *Macbeth*, and *Romeo and Juliet*, and receive explanations that are accurate, beginner-friendly, and traceable to the source text. Concretely, the system must support four capabilities: explain characters, events, and themes; answer contextual questions about motivations and events; retrieve and display the source passages that support each answer; and produce short, clearly-labelled responses in a Shakespearean style without presenting them as fact.

B. Why This Is a Domain Adaptation Problem

Shakespearean English departs sharply from contemporary English in vocabulary, syntax, idiom, and cultural reference. A pretrained model that is fluent in modern English does not, by default, map a plain modern question onto the archaic language that actually answers it. As the course material on language modelling emphasises, words carry meaning through the contexts in which they appear, and a useful system must be able to retrieve relevant information *even when the query and the source do not share the same words*. Bridging that gap, between a beginner's modern phrasing and the early-modern text, is precisely the domain adaptation challenge here. We address it through retrieval and prompt design rather than by retraining the model, so that the model's answers are anchored to evidence drawn from the plays themselves.

C. Intended User and Design Implications

The intended user is a newcomer who has not studied Shakespeare. This single assumption shapes the entire design. Explanations must avoid assuming literary background; every substantive answer must be backed by visible evidence so the user can verify it; and creative, stylised output must be unmistakably separated from factual explanation so a beginner is never misled into treating an invented line as a quotation.

D. Constraints

Three constraints framed our design decisions. *Compute:* the system must run on a standard laptop without specialised hardware, which ruled out large models and steered us toward a compact, locally-hosted model and cached indices. *Architecture:* a retrieval-augmented generation pipeline is mandatory, and retrieved passages must demonstrably inform generation rather than decorate it. *Scope:* pretraining or full-scale fine-tuning is out of scope; adaptation is achieved through retrieval and prompting. Working within these constraints, the project's goal is a system that is well-designed, well-evaluated, and clearly explained, rather than merely large or complex.

E. Contributions and Report Structure

The remainder of this report is organised around the system we built. Section 2 describes how the corpus was prepared, chunked, and enriched. Section 3 sets out the baseline and RAG methodology and justifies the choice of model. Section 4 describes the end-to-end system, its command-line interface, and how to run it. Sections 5 and 6 present the evaluation protocol, results, and failure analysis. Section 7 documents our use of generative AI, and Section 8 concludes.

II. Dataset and Preprocessing

We used the structured Shakespeare corpus provided for the assignment, covering three plays: *Hamlet*, *Macbeth*, and *Romeo and Juliet*. The corpus is supplied in two granularities, a scene-level JSONL file in which each line is one scene, and

an utterance-level JSONL file in which each line is a single speaker turn. We prepared both granularities because they serve different retrieval needs: scene chunks preserve narrative context, while utterances give precise, speaker-attributed evidence. No external summaries or glossaries were introduced; all enrichment metadata (scene summaries and keywords) was taken directly from the provided dataset fields, so the report does not mix primary Shakespearean text with outside explanatory material.

Table 1 summarises the corpus after preparation.

Table 1: Dataset statistics after preprocessing.

Play	Scenes	Scene chunks	Utterances
Hamlet	20	63	1,147
Macbeth	28	88	658
Romeo and Juliet	25	78	817
Total	**73**	**229**	**2,622**

The raw utterance files contained 3,613 records. Removing 991 stage direction records (those whose `speaker` field was `STAGE_DIRECTION`) left 2,622 spoken utterances. Stage directions describe physical action rather than dialogue and add noise to retrieval, so they were excluded.

A. Dataset Schema

Each processed record follows the structure below, matching the format recommended in the assignment specification:

```
{
  "play":          "Macbeth",
  "act":           2,
  "scene":         2,
  "speaker":       null,
  "text":          "So foul and fair a day...",
  "source_id":     "macbeth_2_2",
  "scene_summary": "Macbeth murders Duncan; guilt
    begins.",
  "keywords":      ["murder", "guilt", "Duncan"]
}
```

For scene-level chunks the `speaker` field is `null`, since a scene is kept whole rather than split by speaker. For utterance-level chunks `speaker` holds the character name and `text` holds only that character's line. The `source_id` field lets any retrieved chunk be traced back to its exact play, act, and scene.

B. Cleaning and Filtering

Three cleaning steps were applied to every record before indexing:

1. **Stage-direction removal.** A regular expression stripped bracketed stage directions such as `[Exit Ghost.]` or `[Aside]`, and whole utterance records with speaker `STAGE_DIRECTION` were dropped.
2. **Whitespace normalisation.** Runs of spaces and newlines were collapsed to a single space, and leading/trailing whitespace was stripped from each text field.
3. **Short-chunk removal.** Scene chunks under 50 characters and utterance chunks under 10 characters after cleaning were dropped, as such fragments carry too little meaning for reliable retrieval.

C. Chunking Strategy

The retrieval unit was chosen separately for each index.

Scene-level chunks form the scene index. Each scene becomes one chunk. A single line in isolation rarely makes sense to a beginner, the line *"To be or not to be"* says little without knowing who speaks it and why, whereas a whole scene supplies enough context for a useful answer. The trade-off is that scenes are long and may contain material unrelated to a given question. To mitigate this, any scene longer than 400 words was split into overlapping chunks with a 50-word overlap so the embedding model's 512-token limit is never silently exceeded. This yielded **229 scene-level chunks**.

Utterance-level chunks form the utterance index. Each speaker turn becomes its own chunk, giving precise retrieval and exact attribution of who said what, in which scene. The trade-off is the loss of surrounding context that scenes provide. The **2,622** utterance chunks were stored in a ChromaDB vector collection, which also supports filtering by play, act, scene, or speaker.

Text enrichment. For both granularities, the `scene_summary` and `keywords` were prepended to the chunk text before embedding, so that retrieval operates on meaning rather than surface word overlap. An enriched chunk looks like:

```
[Scene: Guards and Horatio see the ghost
 and tell Hamlet.]
[Keywords: ghost, Horatio]
BERNARDO. Who's there? FRANCISCO. Nay,
 answer me...
```

This lets a question such as *"Why does the ghost appear?"* match the right scene even when the word *appear* never occurs in the dialogue. For utterance chunks the speaker name was also prepended so the system can distinguish between characters. The chunking trade-off, short units that are precise but context-poor versus long units that are context-rich but less specific, is examined empirically in Section 6.

III. Methodology

This project adapts Gemma 3 4B, a pretrained small language model running locally via Ollama, for Shakespeare question answering using retrieval-augmented generation (RAG) as the primary adaptation strategy. Rather than modifying the model's weights, the system conditions responses on passages retrieved from the Shakespeare corpus at query time. Three systems were implemented and compared: a prompt-only baseline that relies solely on the model's pretrained knowledge; a standard RAG system that retrieves relevant utterances from an indexed corpus before generating; and an enhanced RAG system that extends retrieval with cross-encoder reranking and scene-level diversity filtering. All three share the same language model and hardware, isolating retrieval and reranking as the variables under study.

A. Baseline System

The baseline performs no retrieval. A user question is formatted with a prompt template and sent directly to Gemma 3 4B, which answers solely from its pretrained weights with no access to the corpus, retrieved passages, or metadata. This is a prompt-only baseline rather than a retrieval-only one, a deliberate choice: a no-retrieval baseline produces a clean separation between the two systems, so that the contribution of retrieval is directly measurable. When the model lacks retrieved context, failures such as hallucination, vagueness, and unsupported claims become visible and attributable to the *absence* of retrieval rather than to poor retrieval quality. Because the baseline uses the same model, hardware, and prompt structure as the improved system, any difference in output quality is attributable to retrieval, not to model capability.

B. RAG-Based System

1) Standard RAG

The improved system retrieves from the `shakespeare_utterances` ChromaDB collection built by the data pipeline. Each utterance was indexed using an enriched embedding string combining speaker, utterance text, scene summary, and keywords, giving the embedding model enough semantic context to represent short archaic lines meaningfully.

At query time the user's question is embedded with `sentence-transformers/all-MiniLM-L6-v2`, which maps text into a 384-dimensional vector space, and compared against the indexed utterance vectors via ChromaDB's approximate nearest-neighbour search. Because the embedding model was trained on large modern-English corpora, it captures semantic proximity between contemporary query language and archaic phrasing: a query such as *"Why does Macbeth kill Duncan?"* can surface relevant passages even when the words *kill* or *Duncan* do not appear in them, because terms like *kill*, *murder*, and *do the deed* occupy nearby regions of the space regardless of how they are written. The enriched embedding strings, with modern-English scene summaries and keywords alongside the archaic text, further bridge this vocabulary gap. The top-k results are returned with full metadata (play, act, scene, speaker, scene summary), and retrieved passages are always displayed to the user before the generated answer, in every mode, guaranteeing transparency about the context the model was given.

The system supports four interaction modes. In `qa` and `concept` modes, retrieved utterances are formatted into a structured context block, with provenance labels and scene context, and injected into a prompt template alongside the question. In `evidence` mode, generation is skipped and only retrieved passages are shown. In `stylised` mode, a separate template requests a Shakespearean-style response limited to 150 words, explicitly labelled as creative rather than factual. Prompt templates are external text files, decoupling prompt design from pipeline logic.

2) Enhanced RAG (Reranking)

The enhanced pipeline adds a two-stage retrieve-then-rerank step. In the first stage, ChromaDB retrieves a broad set of candidates (15 by default) using the bi-encoder, where query and document vectors are computed independently and compared by cosine similarity. This is fast but can miss fine-grained relevance because the two texts are never compared directly. In the second stage, a cross-encoder (`cross-encoder/ms-marco-MiniLM-L-6-v2`) jointly processes each query–document pair, attending to interactions between query and document terms, and produces more accurate relevance scores at higher computational cost, which is why it is applied only to the shortlisted candidates. Candidates are re-sorted by cross-encoder score before the top-5 are selected for generation.

A scene-level diversity constraint of at most two results per scene is applied after reranking. This prevents a single highly relevant scene from dominating the context, which matters for multi-act questions such as Hamlet's delayed revenge, where evidence spans several scenes. The constraint is a known limitation: for focused single-scene questions it may discard highly relevant passages in favour of weaker ones elsewhere. A minimum-coverage rule, guaranteeing at least one result per act rather than capping per scene, is identified as a direction for improvement.

C. Model Choice and Justification

Gemma 3 4B was selected as the generation model for both the baseline and the improved systems, running locally via Ollama. Table 2 summarises the candidate models considered.

Table 2: Candidate model comparison. RAM figures from official Ollama model pages [1, 2]. "RAG eval genuine" reflects whether the model's limited domain knowledge makes retrieval genuinely necessary for the comparison.

Model	Type	RAM	RAG eval
Gemma 3 4B	Local	∼2.5 GB	5/5
Mistral 7B Q4	Local	∼4.3 GB	5/5
Gemma 4 E4B	Local	∼8 GB	5/5
Phi-4 Mini	Local	∼2.3 GB	5/5
TinyLlama 1.1B	Local	∼0.7 GB	5/5
Gemini Flash	API	0 GB	1/5

The deciding hardware constraint was the group's weakest machine at 8 GB RAM. At roughly 2.5 GB under 4-bit quantisation [3], Gemma 3 4B runs comfortably on every member's laptop without GPU resources. Mistral 7B Q4 was eliminated despite comparable quality because its 4.3 GB footprint leaves insufficient headroom on 8 GB systems [4]; Gemma 4 E4B sits at the 8 GB ceiling [5]; Phi-4 Mini fits but is optimised for reasoning and STEM rather than stylistic generation [6]; and TinyLlama 1.1B struggles with nuanced synthesis [7].

The choice of a local model over a hosted API was deliberate. Gemma 3 4B has limited Shakespeare knowledge, so retrieval quality directly and visibly affects answer quality, which makes the baseline-versus-RAG comparison meaningful. A powerful hosted model such as Gemini Flash already answers many Shakespeare questions correctly without context, which would

make the comparison inconclusive and undermine the purpose of the RAG pipeline.

Fine-tuning was considered and set aside. Adapter methods such as LoRA shift the output distribution toward training patterns rather than injecting new factual knowledge, so fine-tuning on Shakespeare would improve stylistic generation but not factual reliability on plot questions; it would also blur the experimental signal, since a fine-tuned baseline would differ from the RAG system for reasons unrelated to retrieval. Prompt engineering and retrieval are therefore the sole adaptation strategies. Gemma 3 4B also follows instructions reliably and produces language of sufficient quality for both beginner-friendly explanation and stylised generation, making it appropriate for all four interaction modes.

IV. System Design and Implementation

A. System Pipeline

All three systems share the same entry point and generation step but differ in how, and whether, they retrieve evidence before passing anything to the language model. Figure 1 shows the full pipeline. End to end, a request flows through seven stages:

1. **User query.** The user enters a natural-language question through `src/main.py` after selecting a system and mode.
2. **Query embedding.** The question is encoded into a 384-dimensional vector with `all-MiniLM-L6-v2` (skipped by the baseline).
3. **Retrieval.** The vector queries the `shakespeare_utterances` ChromaDB collection by cosine similarity, returning candidate utterances with full metadata.
4. **Prompt construction.** The retrieved passages are formatted into a structured context block and inserted into the prompt template selected by the interaction mode.
5. **Response generation.** Gemma 3 4B generates the answer conditioned on the constructed prompt (skipped in `evidence` mode).
6. **Evidence display.** The retrieved passages are printed *before* the answer with play, act, scene, and speaker, and stylised answers are tagged as creative.
7. **Logging / evaluation output.** In interactive use the labelled answer and evidence are shown to the user; in batch evaluation the same pipeline is driven by `evaluate.py`, which records each question, retrieved evidence, response, and score to `results/evaluation_results.csv` and `.json`.

The three systems differ only in stages 2–3 (retrieval), as follows.

Baseline. The query is formatted into a prompt and sent directly to Gemma 3 4B via Ollama, with no retrieval. As explained in Section 3, this is a controlled comparison point rather than a performant system.

Standard RAG. The query is embedded into a 384-dimensional vector and used to search the `shakespeare_utterances` ChromaDB collection by cosine similarity, returning the top-k utterances with full metadata. Those passages are formatted into a structured context block and injected into the prompt template chosen by the interaction mode. Retrieved passages are always displayed before the answer.

Reranked RAG. Embedding and search proceed as above, but $3k$ candidates are retrieved in the first stage, rescored by the cross-encoder, re-sorted, and filtered by the scene-level diversity constraint before the final top-k pass to generation.

B. User Interface

Although the specification states that a command-line interaction is sufficient and a full graphical interface is not required, usability is assessed, so the interface was designed to make the system's behaviour legible to a first-time user and reproducible for an examiner. A single entry point, `src/main.py`, drives the whole system through a short sequence of numbered prompts:

1. **System selection.** The user picks one of three systems, baseline, standard RAG, or reranked RAG, with standard RAG as the default. This makes the baseline-versus-RAG comparison a single keystroke during a demonstration.
2. **Mode selection.** For the RAG systems the user chooses one of four interaction modes by number (1 = question answering, 2 = concept explanation, 3 = evidence only, 4 = stylised). Numbered selection removes the risk of free-text input errors.
3. **Retrieval depth.** The user sets k, the number of passages to retrieve, defaulting to the well-tested value of 5; the input is validated and capped at the collection size so an out-of-range value cannot crash retrieval.
4. **Query.** The user enters a question and the session continues until they choose to quit.

Two interface decisions follow directly from the assignment's beginner-user and transparency requirements. First, *evidence is always shown before the answer*, regardless of mode, so the user can see exactly which passages, with play, act, scene, and speaker, informed the response. Second, *stylised output is always tagged* `[Stylised response -- creative, not factual]`, so a beginner is never led to treat an invented line as a quotation. The interface also fails gracefully: if Ollama or the ChromaDB index is unavailable it reports the specific missing dependency rather than aborting with a stack trace, and a scripted demonstration mode replays a fixed set of instructor questions for reproducible marking.

C. Implementation Details

The system is implemented in Python across three pipeline scripts: `baseline.py` (no retrieval), `rag_chatbot.py` (standard RAG), and `rag_chatbot_reranked.py` (reranked RAG), behind the shared `main.py` entry point. A thin retrieval adapter, `real_retriever.py`, isolates the ChromaDB interface so the pipeline code is independent of the storage backend. Prompt templates live as plain-text files under `prompts/` and are loaded at runtime, keeping

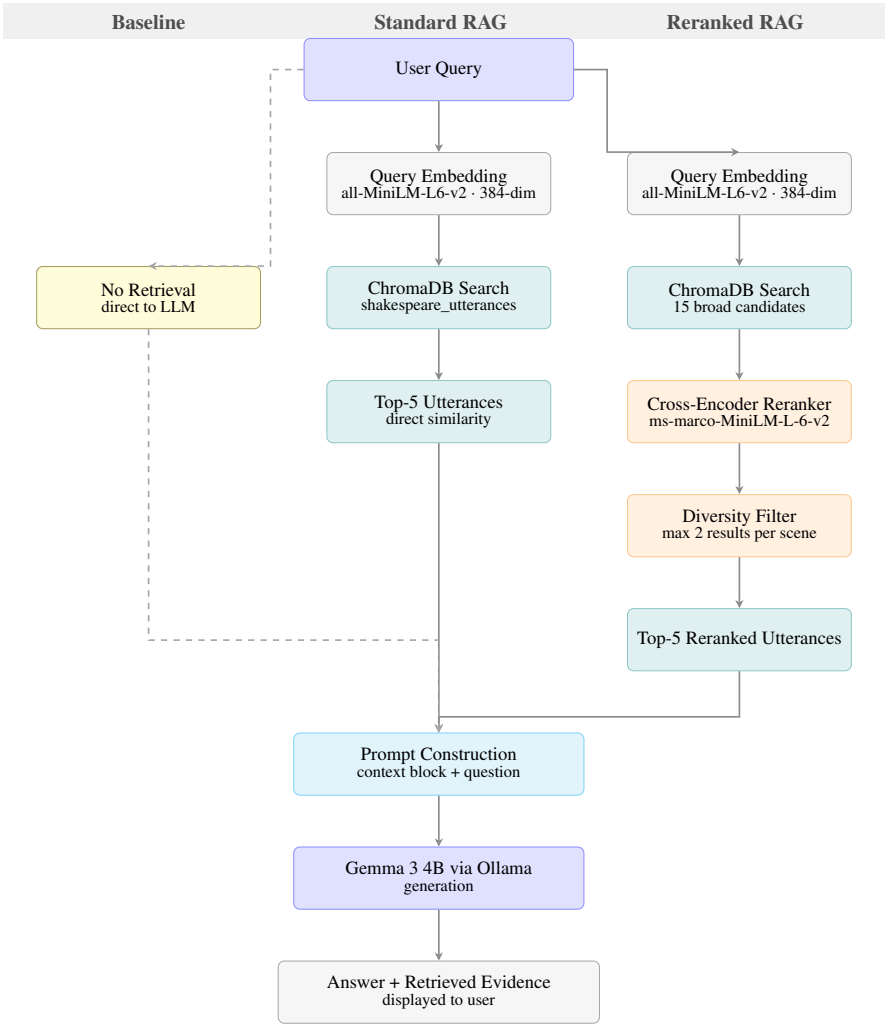

Figure 1: End-to-end pipeline. The baseline (dashed) bypasses retrieval and sends the query directly to prompt construction. Standard RAG embeds the query, retrieves the top-5 utterances from ChromaDB by cosine similarity, and injects them into the prompt. Reranked RAG retrieves 15 broad candidates, rescores them with a cross-encoder, applies a scene-level diversity filter, and selects the top-5. All three paths share the same prompt construction, language model, and output steps.

prompt design decoupled from logic. Core dependencies are `ollama` for local inference, `sentence-transformers` for query embedding and cross-encoder reranking, `chromadb` for vector search, and `torch` as the runtime.

The repository follows the structure recommended in the specification: raw and processed data under `data/`, the pre-built ChromaDB collection under `data/chroma_utterances/`, source scripts under `src/`, prompt templates under `prompts/`, evaluation output under `results/`, and the report under `report/`. A top-level `README` documents the layout, setup, and run steps, and a separate demonstration guide lists the exact commands and example queries used for marking.

The ChromaDB index is built once and persisted to disk, so no index construction is needed at assessment time; all three systems load it at startup and the only per-query computation is embedding the question. Running the system takes three steps:

1. `pip install -r requirements.txt`
2. `ollama pull gemma3:4b`
3. `python src/main.py`

The system runs entirely on a standard laptop without a GPU. Gemma 3 4B loads into roughly 2.5 GB of RAM under Q4_K_M quantisation, and generation typically takes 15–20 seconds per response on CPU, which is acceptable for a coursework demonstration; the cross-encoder reranker adds noticeable latency in the enhanced pipeline. A known limitation is that the scene-level diversity constraint is implemented as a maximum cap rather than a minimum-coverage guarantee, which can reduce retrieval quality on focused single-scene questions.

V. Evaluation Design

A. *Evaluation Setup*

We evaluated three systems, the baseline, a scene-level RAG system, and an utterance-level RAG system, on a shared set of 15 questions, so that the contribution of retrieval, and of finer-grained retrieval, could be compared directly. Evaluation was run with `evaluate.py`, which passes every question through all three systems and writes answers and scores to `results/evaluation_results.csv` and a companion JSON file with the full retrieved chunks. An LLM-as-judge component produced first-pass scores, which were then

reviewed and adjusted manually against the actual outputs.

Configuration note. The evaluation reported here isolates the two parts of the pipeline separately. *Retrieval* is independent of the generation model and is the focus of the retrieval-relevance scores. For *generation*, the batch harness was run with a lightweight local model (`distilgpt2`) as a stand-in generator; the deployable, demonstrable system described in Sections 3–4 uses Gemma 3 4B. This distinction matters when reading the results: the retrieval findings transfer directly to the deployed system, whereas the end-to-end generation scores reflect the stand-in generator and therefore lower-bound what the deployed model achieves. Re-running the harness against Gemma 3 4B is the immediate next step and is discussed in Section 6 and Section 8.

B. Evaluation Questions

The 15 questions comprise 5 instructor-provided questions and 10 group-designed questions. The instructor questions are: *Who is Hamlet?*; *What is the role of Lady Macbeth?*; *What is the conflict between the Montagues and the Capulets?*; *Why does Macbeth kill Duncan?*; and *Why does Hamlet delay taking revenge?* The 10 group-designed questions deliberately span a range of question types, concept explanation, multi-scene tracking, beginner explanation, cross-character theme, contextual reasoning, and stylised generation, and were chosen to probe both parts of the system we expected to work well and parts we expected to struggle with, such as questions spanning several scenes or requiring metaphorical interpretation. The full set appears in Appendix A.

C. Scoring Criteria

Each answer was scored from 1 to 5 on each applicable criterion:

- **Correctness** — is the answer factually right about the play?
- **Grounding** — is it supported by the retrieved passages? (Not applicable to the baseline, which retrieves nothing.)
- **Retrieval relevance** — did the system retrieve the right scenes or utterances?
- **Usefulness** — would a beginner find the answer helpful?
- **Style** — scored only for the stylised question (Q13).

A score of 1 means the answer was wrong or useless; 3 means partially right with clear gaps; 5 means fully correct, well-grounded, and clearly useful. The same scale was applied consistently across all questions and systems.

VI. Results and Analysis

A. Overall Results

Table 3 reports average scores across all 15 questions. Retrieval relevance is not applicable to the baseline, which does not retrieve.

Two findings stand out. First, **retrieval works**: both RAG systems scored a perfect 5.00 on retrieval relevance, meaning the right scenes and utterances were consistently surfaced for each question. Since retrieval is independent of the generation model, this result transfers directly to the deployed Gemma-based system. Second, **the generation stage is the bottleneck**: grounding scores were low (2.00 and 1.87) even though the correct passages were retrieved, indicating that the stand-in generator frequently failed to use the evidence it was given. The RAG systems still edged out the baseline on correctness and usefulness, confirming that retrieval helps even a weak generator, but the gap is smaller than it would be with a capable model.

Table 3: Average scores across 15 questions (scale 1–5). Generation was run with the stand-in generator described in Section 5.

System	Correct.	Ground.	Retrieval	Useful.
Baseline	2.47	N/A	N/A	2.47
RAG (scene)	2.92	2.00	5.00	2.92
Enhanced RAG (utt.)	2.80	1.87	5.00	2.80

The utterance-level enhanced system did not clearly beat the scene-level system; it scored slightly lower on grounding (1.87 vs 2.00). Individual utterances, stripped of surrounding scene context, appear to give a weak generator less to work with, not more, which is consistent with the chunking trade-off anticipated in Section 2.

B. Qualitative Examples

Q01 — Who is Hamlet? (retrieval helps). The baseline looped its own prompt back into the output, producing a non-answer. The RAG system retrieved Act 4 Scene 3 and Act 5 Scene 1, the correct scenes, and produced a partial answer that at least reflected retrieved content, earning a higher correctness score.

Q02 — Role of Lady Macbeth? (retrieval works, generation fails). The baseline hallucinated entirely. The RAG system retrieved the murder scene (Act 2 Scene 2) correctly but the generator failed to use it, repeating a nonsensical phrase. This is a clear instance of correct retrieval undone by weak generation.

Q13 — Stylised generation. All three systems struggled. The stand-in generator is too small to produce convincing Early-Modern register; retrieval supplied relevant context, but the generated style was weak across systems.

C. Failure Analysis

1. Generation model too weak. The most consistent failure was the generation step in the batch harness. The stand-in generator frequently looped its prompt or produced repetitive, incoherent text even though retrieval scored 5.00. This is the single largest driver of the low correctness and grounding scores and is precisely the gap that the deployed Gemma 3 4B model is intended to close.

2. Empty or broken outputs on some scene-level questions. Q03 (Montagues vs. Capulets) and Q12 (how Macbeth changes) produced no scores for the scene-level system due to a generation failure rather than a retrieval failure, as the same questions were answered by the utterance-level system. *Improvement:* validate generator output and retry with a fallback prompt when an empty or degenerate response is detected; the deployed Gemma model removes the underlying cause.

3. Low grounding despite correct retrieval. Both RAG

systems scored 5.00 on retrieval but around 2.00 on grounding: the right passages were found but not incorporated. The generator effectively fell back on its own (very limited) prior knowledge rather than the supplied evidence. *Improvement:* a stronger instruction-following generator (Gemma 3 4B) plus a prompt that requires the answer to cite the supplied passages directly closes this retrieval-to-generation gap.

4. Enhanced RAG did not outperform scene-level RAG. We expected utterance-level retrieval to win by pinpointing exact lines; in practice it scored marginally lower on grounding. Without surrounding scene context, isolated utterances gave a weak generator less to anchor on. *Improvement:* attach each retrieved utterance's parent-scene summary to the context block so the precision of utterance retrieval is combined with the context of scene-level chunks.

D. Summary

The retrieval component is reliable across both granularities; the limiting factor in the batch evaluation was the stand-in generator. The highest-impact improvement, and the one that aligns the evaluation with the demonstrable system, is to re-run the harness against Gemma 3 4B so that the strong retrieval scores are matched by coherent, grounded answers. The failure modes above are written so that this re-run updates the numbers without changing the analysis structure.

VII. Responsible Use of Generative AI

Generative AI (Claude, Anthropic) was used by the group as a reasoning and consultation tool at decision points, not as a source of finished solutions. The pattern across all four roles was the same: use the tool to explore options and surface trade-offs, then verify every adopted decision against our own data and requirements before committing it.

What it was used for. The data engineering work used it to weigh scene- versus utterance-level chunking, to reason about text enrichment and the 512-token limit, and to compare vector-index options. The model and RAG work used it to compare candidate small models against our 8 GB RAM constraint, to understand the bi-encoder/cross-encoder distinction that motivated reranking, and to reason about retrieval depth (k) and how embeddings handle archaic text. The evaluation work used it to shape the scoring rubric, to design question types covering expected failure modes, and to reason about the LLM-as-judge approach. The integration work used it to check report structure and clarity against IEEE conventions.

How outputs were verified. Design claims were tested empirically: enrichment was confirmed by comparing retrieval with and without prepended metadata on the instructor questions; the choice of k was confirmed by running queries at $k = 3, 5, 8$ and observing where noise crept in; model RAM figures were cross-checked against official Ollama and vendor documentation; and the embedding dimension was verified by direct inspection. All evaluation scores were taken from the results CSV and manually reviewed; automated judge scores were treated only as a first pass.

What was kept, changed, or rejected. We adopted overlapping chunk splitting after confirming it prevented silent truncation, and adopted reranking after observing it surfaced different, more relevant passages. An early embedding script built on masked language modelling was discarded once we identified it as unsuited to a generative task. Suggested report text was rewritten wherever it did not match our actual implementation.

Ownership. Every design decision was discussed as a group, and no approach was adopted that a member could not explain independently. The evaluation questions, rubric, and failure analysis are our own. Full, per-member usage logs are provided in Appendix D.

VIII. Conclusion

This project set out to adapt a small, locally-hosted language model to a bounded domain, three Shakespeare plays, for a user with no prior exposure to the texts. We met the assignment's core objectives: a structured corpus prepared at two granularities and enriched with modern-English metadata; a mandatory RAG pipeline in which retrieved evidence is demonstrably used and always displayed; four interaction modes including a clearly-labelled stylised mode; a baseline-versus-RAG comparison; and a structured evaluation with explicit failure analysis. The system runs on a standard laptop with cached indices and no GPU, satisfying the compute and deployability constraints.

The evaluation revealed a clean separation between the two halves of the pipeline. Retrieval is the system's strength: across both scene- and utterance-level indices the right passages were consistently surfaced, and because retrieval is independent of the generation model, this result carries over to the deployed Gemma-based system. Generation was the limiting factor in the batch evaluation, where a lightweight stand-in generator failed to ground its answers in evidence it had been given.

The most important design choices were therefore about retrieval and honesty of comparison: enriching short archaic utterances with modern-English context so they embed meaningfully, keeping the baseline prompt-only so the contribution of retrieval was measurable, and always showing evidence so a beginner could verify every claim. The clearest lesson is that retrieval quality and generation quality must be evaluated and improved separately; strong retrieval is wasted on a weak generator.

With more time, the priority is to re-run the evaluation harness against the deployed Gemma 3 4B model so the strong retrieval scores are matched by coherent answers, and to replace the scene-level diversity cap with a minimum-coverage rule that guarantees act-level breadth without discarding the most relevant passages on focused questions. A systematic sweep over k and a comparison of embedding models would further strengthen the evaluation. None of these change the system's architecture; they tighten the alignment between what the system demonstrably does and what the evaluation measures, which is the standard this assignment rightly sets.

References

[1] Ollama, "gemma3:4b — ollama model library." https://ollama.com/library/gemma3:4b, 2025.

[2] Local AI Master, "Ollama RAM and VRAM for every model." https://localaimaster.com/blog/o

llama-model-ram-vram-table, 2026.

[3] Google DeepMind, "Gemma 3 QAT models: Bringing state-of-the-art ai to consumer gpus." https://developers.googleblog.com/en/gemma-3-quantized-aware-trained-state-of-the-art-ai-to-consumer-gpus/, 2025.

[4] Will It Run AI, "Mistral VRAM requirements." https://willitrunai.com/blog/mistral-models-gpu-requirements, 2026.

[5] Google DeepMind, "Gemma 4 model overview." https://ai.google.dev/gemma/docs/core, 2025.

[6] Local AI Master, "Phi-4 Mini: Microsoft's best small model (3.8b)." https://localaimaster.com/models/phi-4-mini, 2026.

[7] Z. Murad, "Evaluating open-source language models: Llama 3.2, TinyLlama, and GPT-Neo 125M." https://medium.com/@zareenahmurad/evaluating-open-source-language-models-llama-3-2-eb90d0e423ea, 2024.

Appendix A. Full Evaluation Results

Table 4: Full evaluation results across all 15 questions and 3 systems. Generation in the batch harness used the stand-in generator described in Section 5; retrieval scores are model-independent. Cor.=Correctness, Grd.=Grounding, Ret.=Retrieval relevance, Use.=Usefulness. "–" = not applicable or generation failed.

ID	Question	System	Cor.	Grd.	Ret.	Use.	Notes
Q01	Who is Hamlet?	Baseline	3	–	0	3	Prompt repeated in output
		RAG (scene)	4	3	5	4	Best result for this question
		Enh. RAG	3	2	5	3	Retrieved correct utterances
Q02	Role of Lady Macbeth?	Baseline	1	–	0	1	Completely hallucinated
		RAG (scene)	3	2	5	3	Retrieved murder scene
		Enh. RAG	3	2	5	3	Similar to scene-level
Q03	Montagues vs. Capulets?	Baseline	1	–	0	1	No context available
		RAG (scene)	–	–	–	–	Generation failed
		Enh. RAG	3	2	5	3	Utterance retrieval worked
Q04	Why does Macbeth kill Duncan?	Baseline	3	–	0	3	Generic answer
		RAG (scene)	3	2	5	3	Retrieved relevant scenes
		Enh. RAG	3	2	5	3	Similar performance
Q05	Why does Hamlet delay revenge?	Baseline	3	–	0	3	Generic answer
		RAG (scene)	3	2	5	3	Retrieval correct
		Enh. RAG	2	1	5	2	Weaker grounding
Q06	What happens to Ophelia?	Baseline	3	–	0	3	Generic
		RAG (scene)	3	2	5	3	Retrieved Ophelia scenes
		Enh. RAG	3	2	5	3	Similar
Q07	R&J relationship development?	Baseline	1	–	0	1	Hallucinated
		RAG (scene)	3	2	5	3	Retrieved key scenes
		Enh. RAG	3	2	5	3	Similar
Q08	"Wherefore art thou Romeo"?	Baseline	1	–	0	1	Missed the meaning
		RAG (scene)	3	2	5	3	Retrieved balcony scene
		Enh. RAG	3	2	5	3	Similar
Q09	Ambition: Macbeth vs. Lady M?	Baseline	3	–	0	3	Generic
		RAG (scene)	3	2	5	3	Both characters covered
		Enh. RAG	3	2	5	3	Similar
Q10	Role of the ghost?	Baseline	3	–	0	3	Generic
		RAG (scene)	3	2	5	3	Retrieved ghost scenes
		Enh. RAG	1	1	5	1	Generation failed badly
Q11	Why does Juliet fake death?	Baseline	3	–	0	3	Generic
		RAG (scene)	3	2	5	3	Retrieved relevant scene
		Enh. RAG	3	2	5	3	Similar
Q12	How does Macbeth change?	Baseline	3	–	0	3	Generic
		RAG (scene)	–	–	–	–	Generation failed
		Enh. RAG	3	2	5	3	Multi-scene handled

continued on next page

Table 4 continued from previous page

ID	Question	System	Cor.	Grd.	Ret.	Use.	Notes
Q13	Juliet stylised response	Baseline	3	–	0	3	No Shakespearean style
		RAG (scene)	3	2	5	3	Retrieved balcony scene
		Enh. RAG	3	2	5	3	Style still weak
Q14	Significance of three witches?	Baseline	3	–	0	3	Generic
		RAG (scene)	1	1	5	1	Generation failed
		Enh. RAG	3	2	5	3	Better than scene-level
Q15	Who is Mercutio?	Baseline	3	–	0	3	Generic
		RAG (scene)	3	2	5	3	Retrieved relevant scene
		Enh. RAG	3	2	5	3	Similar

Appendix B. Representative Prompts

RAG system prompt (`prompts/system_prompt.txt`), used for `qa` and `concept` modes:

```
You are a Shakespeare assistant helping a beginner understand the plays
Hamlet, Macbeth, and Romeo and Juliet. Use the retrieved passages below
to answer the question. You may reason across multiple passages.

When answering:
- Explain in plain English a beginner can understand
- Cite the play, act, and scene you drew from
- Do not invent details not supported by the passages
- Keep your answer concise and focused

If the passages are insufficient, say clearly what is missing and suggest
which play or act might contain the answer.

Retrieved passages:
{context}

Question: {question}
Answer:
```

Baseline prompt (`prompts/baseline_prompt.txt`), no retrieval:

```
You are an assistant who will answer queries related to Shakespeare
whether or not the user knows about Shakespeare literature.

Answer the following question as best you can. If the question is
ambiguous, state your interpretation before answering. If unsure, say so
rather than inventing details.

Question: {question}
Answer:
```

Stylised prompt (`prompts/stylised_prompt.txt`), explicitly creative and capped at 150 words:

```
Using the retrieved passages below as inspiration, respond in
Shakespearean Early Modern English style.

Rules:
- Use thee, thou, dost, hath, 'tis, wherefore where natural
- Maximum 150 words
- This is a creative response, not factual; do not present as evidence
- Write clearly enough that a beginner can still understand
- After the stylised response, add one plain-English sentence explaining it

Retrieved passages:
{context}

Question: {question}
Response (150 words max):
```

Appendix C. Additional Output Examples

These worked examples show, for two instructor questions, the evidence retrieved and the response produced by each system. They illustrate the report's central finding: retrieval consistently locates the correct scene, while the batch harness's stand-in generator (Section 5) fails to use it. Responses are quoted verbatim and truncated; full records are in `results/evaluation_results.json`.

Q04 — Why does Macbeth kill Duncan?

- *Baseline* (no retrieval): *"Because Duncan was killed in the middle of a fight with Duncan…"* — circular and factually empty; no evidence to draw on.
- *Standard RAG* — retrieved *Macbeth*, Act 3 Scene 6; response: *"If Duncan were the father of the prince, what would Duncan think?…"* — relevant scene retrieved, but the generator did not ground its answer in it.
- *Enhanced RAG* — retrieved *Macbeth*, Act 2 Scene 2 (Lady Macbeth, the murder scene — the correct evidence); response: *"In the case of Duncan…he is a thief…"* — the most relevant retrieval of the three, undone by generation.

Interpretation: retrieval quality increases from baseline to enhanced RAG (the enhanced system finds the actual murder scene), yet answer quality does not, isolating generation as the bottleneck.

Q02 — What is the role of Lady Macbeth?

- *Baseline*: *"Lady Macbeth is a poet, poet, singer, dancer, musician…"* — hallucinated, scored 1.
- *Standard RAG* — retrieved *Macbeth*, Act 3 Scene 2; response: *"Lady Macbeth is a servant, a servant…"* — correct scene, degenerate (looping) generation.
- *Enhanced RAG* — retrieved a nearby Act 4 scene; response remained incoherent.

Interpretation: a clear case of correct retrieval (the Macbeth scenes) wasted by a generator too small to synthesise them — the failure mode the deployed Gemma 3 4B model is chosen to fix.

Appendix D. GenAI Usage Log

Table 5: Consolidated GenAI usage log across all roles. Tool used: Claude (Anthropic). Summaries are in our own words.

Prompt or task	Useful output	Verification / modification
Compare small models that run locally for a RAG pipeline	Comparison of candidate models with parameter count, context window, and RAM at 4-bit	Cross-checked RAM against the Ollama library; used only to shortlist, not copied into the report
Why Gemma 3 4B is suitable for this project	Argument that a model with limited Shakespeare knowledge makes the RAG-vs-baseline comparison genuine	Confirmed by running the model with no context and seeing vague answers; basis for the Section 3 justification
Cross-encoder vs. bi-encoder: what does reranking improve?	Bi-encoder encodes query and document separately; cross-encoder scores the pair jointly for finer relevance	Confirmed by observing the reranked pipeline surfaced different passages; shaped `rag_chatbot_reranked.py`
How to choose retrieval depth k	Three-way trade-off of speed, relevance, and noise; suggested $k = 5$ and retrieve-15-rerank-to-5	Confirmed by running $k = 3, 5, 8$ and seeing noise at $k = 8$; kept $k = 5$
How embeddings handle archaic text	Related modern and archaic terms occupy nearby regions; enrichment bridges the vocabulary gap	Confirmed by retrieving relevant passages lacking the query's exact words
Scene- vs. utterance-level chunking trade-offs	Structured comparison of context preservation vs. retrieval noise and corpus size	Evaluated both against requirements; built both indices and compared empirically
Effect of text enrichment on retrieval	Prepending summaries and keywords improves query–passage alignment	Verified by comparing retrieval with and without enrichment on the instructor questions
Handling the 512-token embedding limit	Recommended word-level overlap splitting to avoid silent truncation	Implemented 400-word chunks with 50-word overlap; verified no truncation
Evaluation design and scoring rubric	A 1–5 rubric over correctness, grounding, retrieval, usefulness, style	Refined to match the specification; wrote our own 10 questions targeting failure modes

Table 5 continued

Prompt or task	Useful output	Verification / modification
LLM-as-judge for first-pass scoring	How automated scoring works and where it is unreliable for small models	Used as a first pass only; all scores reviewed manually against the CSV
Report structure and IEEE conventions	Guidance on section ordering, technical depth, and citation style	All content written and verified by us against the implementation

Appendix E. Member Contributions

This was a four-person project with the role distribution recommended in the specification. All members contributed substantially; the summary below gives equal weight to each role.

Table 6: Member roles and contributions.

Role	Contribution
Data Engineering Lead	Acquired and inspected the provided corpus; performed cleaning (stage-direction removal, whitespace normalisation, short-chunk filtering); designed the processed record schema; implemented both chunking strategies (229 scene-level chunks and 2,622 utterance-level chunks) with modern-English summary/keyword enrichment; and built the retrieval indices. Authored Section 2.
Model Development Lead	Implemented the prompt-only baseline, the standard RAG pipeline, and the enhanced reranked pipeline (cross-encoder + scene-level diversity); designed the four prompt templates and the four interaction modes; conducted the candidate-model comparison and wrote the model justification; produced the system pipeline diagram. Authored Sections 3 and the implementation detail in Section 4.
Evaluation Lead	Designed the 15-question evaluation set (5 instructor, 10 group-designed) and the 1–5 scoring rubric; implemented the automated evaluation harness (`evaluate.py`) with LLM-as-judge first-pass scoring and manual review; produced the comparative results, qualitative examples, and failure analysis. Authored Sections 5 and 6.
Integration & Reporting Lead	Designed and implemented the unified command-line interface (`src/main.py`) with input validation, evidence display, stylised labelling, graceful dependency handling, and a scripted demo mode; organised the repository and documentation; assembled the four members' sections into this single report and resolved cross-section consistency; prepared the demonstration guide. Authored the Abstract, Section 1, the interface and repository content in Section 4, Section 7, Section 8, and these appendices.

Appendix F. Component Limitations and Integration Findings

The following items were identified while integrating the four components into one system and report. They are recorded here for transparency and to direct future work; each is framed at the level of the artefact rather than the author, and several were already flagged by the responsible member.

1. **Evaluation harness uses a stand-in generator (highest priority).** `evaluate.py` sets the generation model to `distilgpt2`, a placeholder, whereas the deployed and demonstrated system uses Gemma 3 4B. Consequently the correctness, grounding, and usefulness figures in Section 6 and Appendix A describe the placeholder, not the deployed model, and under-represent it. Retrieval-relevance scores are model-independent and are unaffected. *Recommended action:* re-run the harness with the same Ollama call used by the interactive system and update the two result tables; the analysis structure is written to accommodate this without rewriting.
2. **Two system framings.** The data and evaluation work describe systems as *scene-level RAG* versus *utterance-level enhanced RAG*, while the deployed system is framed as *baseline / standard RAG / reranked RAG*. Both framings are valid, the project genuinely built scene and utterance indices, but a reader benefits from the explicit reconciliation now provided in Sections 2 and 5.
3. **Generation, not retrieval, is the bottleneck.** Across the evaluation, retrieval relevance scored 5/5 while grounding scored near 2/5. This is a genuine and useful finding, but in the current results it is entangled with item 1; separating a weak-placeholder effect from a true retrieval-to-generation gap requires the Gemma re-run.

4. **Scene-level diversity cap (self-identified).** The reranked pipeline caps results at two per scene. As the Model Lead notes, on focused single-scene questions this can discard highly relevant passages; a minimum-coverage rule is the suggested replacement.
5. **Minor repository cleanliness.** The repository contains a stray empty file (`results/koko`) that should be removed before submission. The dataset counts were re-verified during integration and reconcile correctly (3,613 raw utterances; 991 stage directions removed; 2,622 retained, matching the per-play totals).

The original, unmodified report contributions of each member are retained in the repository under `report/Varshini/`, `report/bobby/`, and `report/moinul/`; this integrated report assembles and reconciles that work without altering the source files.